```
// ========================================
// ColdChain Sentinel Prototype
// ESP32 + DHT11 + MQ135 + LCD + Relays
// ========================================

// | Component   | GPIO    |
// | ----------- | ------- |
// | DHT11 Room1 | GPIO 4  |
// | DHT11 Room2 | GPIO 5  |
// | MQ135       | GPIO 34 |
// | Relay 1     | GPIO 18 |
// | Relay 2     | GPIO 19 |
// | Relay 3     | GPIO 21 |
// | Red LED     | GPIO 22 |
// | Green LED   | GPIO 23 |
// | LCD SDA     | GPIO 26 |
// | LCD SCL     | GPIO 27 |

#include <WiFi.h>
#include <HTTPClient.h>
#include <ArduinoJson.h>
#include <DHT.h>
#include <LiquidCrystal_I2C.h>
#include <Wire.h>

// ========================
// WIFI CONFIGURATION
// ========================
const char* ssid = "Monchel";
const char* password = "Monchel1236";

// REPLACE "YOUR_COMPUTER_IP_ADDRESS" with the local IP of the PC running the
backend
const char* backendUrl = "http://192.168.202.125:5000/api/telemetry";

// ========================
// DHT CONFIGURATION
// ========================
#define DHTTYPE DHT11

#define DHT_ROOM1_PIN 4
#define DHT_ROOM2_PIN 5

DHT dhtRoom1(DHT_ROOM1_PIN, DHTTYPE);
DHT dhtRoom2(DHT_ROOM2_PIN, DHTTYPE);
```

```
// ========================
// RELAY PINS
// ========================
#define COOL_FAN_ROOM1 18
#define COOL_FAN_ROOM2 19
#define WARM_FAN_ROOM2 21

// ========================
// LED PINS
// ========================
#define RED_LED 22
#define GREEN_LED 23

// ========================
// MQ135 PIN
// ========================
#define MQ135_PIN 34

// ========================
// LCD CONFIGURATION
// ========================
LiquidCrystal_I2C lcd(0x27, 16, 2);

// Target Temps
float room1Min = 2.0;
float room1Max = 8.0;
float room2Min = 0.0;
float room2Max = 2.0;

// Manual Controls from Web
bool r1Sys = true;
bool r1Fan = false;
bool r1Comp = true;

bool r2Sys = true;
bool r2Fan = false;
bool r2Comp = true;

// ========================
// VARIABLES
// ========================
float realTemp1, hum1;
float realTemp2, hum2;
```

```
float temp1;
float temp2;

int airQuality;

String alertMessage = "SAFE";

// ========================
// SETUP
// ========================
void setup() {

  Serial.begin(115200);

  // Start WiFi
  Serial.print("Connecting to WiFi");
  WiFi.begin(ssid, password);
  while (WiFi.status() != WL_CONNECTED) {
    delay(500);
    Serial.print(".");
  }
  Serial.println("\nConnected to WiFi");
  Serial.print("ESP32 IP: ");
  Serial.println(WiFi.localIP());

  // Start DHT Sensors
  dhtRoom1.begin();
  dhtRoom2.begin();

  // Relay Pins
  pinMode(COOL_FAN_ROOM1, OUTPUT);
  pinMode(COOL_FAN_ROOM2, OUTPUT);
  pinMode(WARM_FAN_ROOM2, OUTPUT);

  // LED Pins
  pinMode(RED_LED, OUTPUT);
  pinMode(GREEN_LED, OUTPUT);

  // MQ135
  pinMode(MQ135_PIN, INPUT);

  // LCD I2C
  Wire.begin(26, 27);

  lcd.init();
```

```
  lcd.backlight();

  // Initial Relay States
  digitalWrite(COOL_FAN_ROOM1, LOW);
  digitalWrite(COOL_FAN_ROOM2, LOW);
  digitalWrite(WARM_FAN_ROOM2, LOW);

  // Welcome Screen
  lcd.setCursor(0, 0);
  lcd.print("ColdChain");
  lcd.setCursor(0, 1);
  lcd.print("Sentinel");

  delay(3000);

  lcd.clear();
}

void sendTelemetryToBackend(float realT1, float realH1, float realT2, float
realH2, float t1, float h1, float t2, float h2, int aqi) {
  if(WiFi.status() == WL_CONNECTED){
    HTTPClient http;

    http.begin(backendUrl);
    http.addHeader("Content-Type", "application/json");

    // Average of both DHT sensors = ambient room conditions
    float avgTemp = (realT1 + realT2) / 2.0;
    float avgHum  = (realH1 + realH2) / 2.0;

    // Build JSON payload
    // room.ip        → matches StorageRoom.ipAddress in DB (192.168.1.20)
    // fridges[0].ip  → matches Device.ipAddress for Main Fridge (192.168.1.10)
    // fridges[1].ip  → matches Device.ipAddress for Vaccine Freezer
(192.168.1.11)
    String payload = "{";
    payload += "\"secret\":\"supersecretkey123\",";
    payload += "\"room\":{";
    payload += "\"ip\":\"192.168.1.20\",";
    payload += "\"airQuality\":" + String(aqi) + ",";
    payload += "\"ambientTemp\":" + String(avgTemp, 1) + ",";
    payload += "\"ambientHum\":"  + String(avgHum,  1);
    payload += "},";
    payload += "\"fridges\":[";
```

```
    payload += "{\"ip\":\"192.168.1.10\",\"temperature\":" + String(t1, 1) +
",\"humidity\":" + String(h1, 1) + "},";
    payload += "{\"ip\":\"192.168.1.11\",\"temperature\":" + String(t2, 1) +
",\"humidity\":" + String(h2, 1) + "}";
    payload += "]}";

    Serial.println("Sending telemetry to backend...");
    int httpResponseCode = http.POST(payload);

    if(httpResponseCode > 0){
      Serial.print("Telemetry HTTP response code: ");
      Serial.println(httpResponseCode);

      String response = http.getString();
      Serial.print("Backend response: ");
      Serial.println(response);

      // Parse JSON from Backend
      DynamicJsonDocument doc(1024);
      DeserializationError error = deserializeJson(doc, response);

      if (!error) {
        Serial.println("Telemetry sent successfully and response parsed.");
        if (doc["config"]) {
          Serial.println("Backend returned configuration updates.");
          if (doc["config"]["192.168.1.10"]) {
            room1Min = doc["config"]["192.168.1.10"]["min"];
            room1Max = doc["config"]["192.168.1.10"]["max"];
            r1Sys    = doc["config"]["192.168.1.10"]["sys"];
            r1Fan    = doc["config"]["192.168.1.10"]["fan"];
            r1Comp   = doc["config"]["192.168.1.10"]["comp"];
          }
          if (doc["config"]["192.168.1.11"]) {
            room2Min = doc["config"]["192.168.1.11"]["min"];
            room2Max = doc["config"]["192.168.1.11"]["max"];
            r2Sys    = doc["config"]["192.168.1.11"]["sys"];
            r2Fan    = doc["config"]["192.168.1.11"]["fan"];
            r2Comp   = doc["config"]["192.168.1.11"]["comp"];
          }
        } else {
          Serial.println("No config object returned by backend.");
        }
      } else {
        Serial.print("Failed to parse backend response: ");
        Serial.println(error.c_str());
```

```
      }
    } else {
      Serial.print("Error sending to backend: ");
      Serial.println(httpResponseCode);
    }

    http.end();
  } else {
    Serial.println("Error in WiFi connection");
  }
}

// =========================
// LOOP
// =========================
void loop() {

  // =========================
  // READ REAL SENSOR VALUES
  // =========================

  realTemp1 = dhtRoom1.readTemperature();
  hum1 = dhtRoom1.readHumidity();

  realTemp2 = dhtRoom2.readTemperature();
  hum2 = dhtRoom2.readHumidity();

  // =========================
  // SIMULATE FRIDGE TEMPERATURES
  // Convert:
  // 24°C - 28°C  -->  2°C - 8°C
  // =========================

  temp1 = (realTemp1 - 24.0) * (8.0 - 2.0) / (28.0 - 24.0) + 2.0;

  // Convert:
  // 24°C - 28°C  -->  0°C - 2°C

  temp2 = (realTemp2 - 24.0) * (2.0 - 0.0) / (28.0 - 24.0) + 0.0;

  // =========================
  // READ AIR QUALITY
  // =========================

  // airQuality = analogRead(MQ135_PIN);
```

```
airQuality = 302;
 // ========================
 // SERIAL MONITOR
 // ========================

 Serial.println("================================");

 Serial.print("Real Temp Room1: ");
 Serial.print(realTemp1);
 Serial.println(" C");

 Serial.print("Simulated Room1 Temp: ");
 Serial.print(temp1);
 Serial.println(" C");

 Serial.print("Room1 Humidity: ");
 Serial.print(hum1);
 Serial.println(" %");

 Serial.println("--------------------------------");

 Serial.print("Real Temp Room2: ");
 Serial.print(realTemp2);
 Serial.println(" C");

 Serial.print("Simulated Room2 Temp: ");
 Serial.print(temp2);
 Serial.println(" C");

 Serial.print("Room2 Humidity: ");
 Serial.print(hum2);
 Serial.println(" %");

 Serial.println("--------------------------------");

 Serial.print("Air Quality: ");
 Serial.println(airQuality);

 // ========================
 // ROOM 1 CONTROL (Refrigerator)
 // ========================

 if (!r1Sys) {
   digitalWrite(COOL_FAN_ROOM1, LOW); // System off
 } else {
```

```
  // If manual Fan is ON, or (Compressor is ON and temp is too high)
  if (r1Fan || (r1Comp && temp1 > room1Max)) {
    digitalWrite(COOL_FAN_ROOM1, HIGH);
  } else {
    digitalWrite(COOL_FAN_ROOM1, LOW);
  }
}

// ========================
// ROOM 2 CONTROL (Freezer)
// ========================

if (!r2Sys) {
  digitalWrite(COOL_FAN_ROOM2, LOW);
  digitalWrite(WARM_FAN_ROOM2, LOW);
} else {
  bool coolOn = r2Fan; // Fan switch forces circulation (cooling fan)
  bool warmOn = false;

  if (r2Comp) {
    if (temp2 > room2Max) {
      coolOn = true;
      warmOn = false;
    } else if (temp2 < room2Min) {
      coolOn = false;
      warmOn = true;
    }
  }
  digitalWrite(COOL_FAN_ROOM2, coolOn ? HIGH : LOW);
  digitalWrite(WARM_FAN_ROOM2, warmOn ? HIGH : LOW);
}

// ========================
// ALERT DETECTION
// ========================

bool danger = false;

alertMessage = "SAFE";

// ROOM 1 ALERTS

if (temp1 > room1Max) {

  danger = true;
```

```
  alertMessage = "R1 HIGH TEMP";

}
else if (temp1 < room1Min) {

  danger = true;
  alertMessage = "R1 LOW TEMP";
}

// ROOM 2 ALERTS

else if (temp2 > room2Max) {

  danger = true;
  alertMessage = "R2 HIGH TEMP";
}
else if (temp2 < room2Min) {

  danger = true;
  alertMessage = "R2 LOW TEMP";
}

// AIR QUALITY ALERT

else if (airQuality > 2500) {

  danger = true;
  alertMessage = "BAD AIR";
}

if (!r1Sys && !r2Sys) {
  alertMessage = "SYS STOPPED";
  danger = false; // Override danger LED if purposefully stopped
}

// ========================
// LED STATUS
// ========================

if (danger) {

  digitalWrite(RED_LED, HIGH);
  digitalWrite(GREEN_LED, LOW);

}
```

```
else {

  digitalWrite(RED_LED, LOW);
  digitalWrite(GREEN_LED, HIGH);
}

// ========================
// PRINT ALERT STATUS
// ========================

Serial.print("System Status: ");
Serial.println(alertMessage);

// ========================
// LCD DISPLAY — 4 screens, 4 seconds each
// ========================

if (!r1Sys && !r2Sys) {
  // ── System stopped ──────────────────────
  lcd.clear();
  lcd.setCursor(0, 0);
  lcd.print("SYSTEM STOPPED");
  lcd.setCursor(0, 1);
  lcd.print("All units OFF");
  delay(4000);

} else {

  // ── Screen 1: Fridge 1 temp & humidity ──────
  lcd.clear();
  lcd.setCursor(0, 0);
  lcd.print("Fridge 1");
  if (!r1Sys) {
    lcd.setCursor(0, 1);
    lcd.print("System OFF");
  } else {
    lcd.setCursor(0, 1);
    lcd.print("T:");
    lcd.print(temp1, 1);
    lcd.print("C H:");
    lcd.print(hum1, 0);
    lcd.print("%");
  }
  delay(4000);
```

```
    // ── Screen 2: Fridge 2 temp & humidity ──────
    lcd.clear();
    lcd.setCursor(0, 0);
    lcd.print("Fridge 2");
    if (!r2Sys) {
      lcd.setCursor(0, 1);
      lcd.print("System OFF");
    } else {
      lcd.setCursor(0, 1);
      lcd.print("T:");
      lcd.print(temp2, 1);
      lcd.print("C H:");
      lcd.print(hum2, 0);
      lcd.print("%");
    }
    delay(4000);

    // ── Screen 3: Air quality (MQ135 raw ADC) ───
    lcd.clear();
    lcd.setCursor(0, 0);
    lcd.print("Air Quality");
    lcd.setCursor(0, 1);
    lcd.print("MQ135: ");
    lcd.print(airQuality);
    delay(4000);

    // ── Screen 4: System status / alert ─────────
    lcd.clear();
    lcd.setCursor(0, 0);
    if (danger) {
      lcd.print("!! ALERT !!");
    } else {
      lcd.print("Status: OK");
    }
    lcd.setCursor(0, 1);
    lcd.print(alertMessage);
    delay(4000);
  }

  // SEND REAL-TIME DATA TO DASHBOARD
  sendTelemetryToBackend(realTemp1, hum1, realTemp2, hum2, temp1, hum1, temp2,
hum2, airQuality);
}
```